AI-based PV Detection from Electricity Consumption Profiles

Team AI-uto!: Giorgio Ripani, Leonardo Lodolini, Andrea Gallo, Flavio Davide Borriello, Francesco Tassi

October 18, 2026

1 Objective

Aziende Industriali di Lugano (AIL) is a Swiss energy utility that proposed the following challenge: "How much can we infer about a photovoltaic (PV) installation—its presence, commissioning date and nominal capacity—by observing only the evolution of the customer electricity consumption profile?" AIL records how much electricity each customer takes from the grid every 15 minutes. From that curve alone, and without weather or location data, the goal was to say for every customer (1) whether a PV system is installed, (2) when it was switched on, and (3) its nominal capacity in kWp.

Our solution is built on one effect: Once the panels are on the roof, the customer buys much less electricity around noon, and the drop is greater on sunny summer days. We built three simple statistical models, one per goal, each incorporating a different statistical approach. The first estimates the probability that a customer has a PV system; if it is above 0.5, the second estimates the installation date and the third the nominal capacity. In leave-one-customer-out validation the whole pipeline scores **81.8** points, against 54.6 for a trivial baseline.

2 Approach

2.1 Why simple models

We had 40 labeled customers, 22 of them with PV (plus 1147 unlabeled curves, used for checks and one pseudo-labeling experiment, not in the final model). With so few examples, one customer more or less moves the score by about 2 points, and anything with many parameters (deep networks, large forests) just memorizes the 40 customers. So every model below has few features and can be simply explained. The rules also shaped the project: no weather, no location, no sun irradiation data were allowed to be used. We therefore use fixed UTC hour bands (from `timestamp_epoch`): night 0–4, morning 6–8, midday 10–13, afternoon 16–19 UTC. Working in UTC days also removes the 92- and 100-interval days caused by daylight saving time.

2.2 Architecture and frameworks

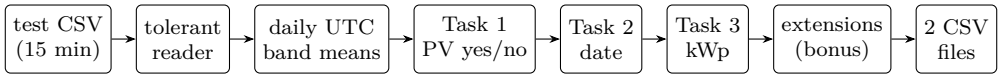


Figure 1: `inference.py`. Each block has a fallback (PV = yes, first day of data, median kWp), so both output files are always written with one row per customer.

Everything is plain Python 3.12: `pandas` and `DuckDB` for loading (the 7.9 GB unlabelled file does not fit in memory), `numpy`/`scipy` for the models, `scikit-learn` and `ruptures` for one change-point feature, `pytest` for 214 tests. The trained parameters are two small JSON files.

2.3 Task 1 – is there PV?

For every summer day (May–August) we measure the midday dip, $\text{dip}_d = \log \frac{\text{midday}_d+\varepsilon}{(\text{morning}_d+\text{afternoon}_d)/2+\varepsilon}$. Our first idea was the average dip, but some customers without PV also consume less at midday. The key observation is that a PV dip *changes with the clouds*, while a dip caused by habits does not. So we use its spread, $\texttt{dip_spread} = q_{0.9} - q_{0.1}$ over summer days, with a threshold.

Systems installed a few months before the end of the data have no summer to look at, so customers rejected by the first rule get a second check, `rec_drop`: the median dip of the last 90 days against the same days one year earlier. Outstandingly, both features use only the *last two years* of each customer, although test series are about four years long. This counter-intuitive method aims at filtering out old changes in habits which impact energy consumption. If a PV is present in fact, the weather variability of the two summers is enough to predict its presence and a plant installed in an earlier period stays visible also in the last two years.

2.4 Task 2 – commissioning date

We fit the daily log ratio $y_t = \log(\text{midday}_t/\text{night}_t)$ with a seasonal curve to absorb normal season swing plus a step:

$$y_t = a + b\sin\tfrac{2\pi t}{365.25} + c\cos\tfrac{2\pi t}{365.25} + s\,\mathbf{1}[t \geq \tau] + \epsilon_t,$$

trying every candidate day as τ and keeping the one with the smallest squared error. The night never sees the sun, so it works as a reference for how much the customer consumes. If no step improves the error by at least 5%, a value fixed in advance and confirmed through testing, we assume the panels were already there and we return the first day. No parameter is trained on labels.

2.5 Task 3 – capacity

From the days before the estimated installation date, we derive the customer's typical midday-to-night consumption ratio r. For each summer day after installation, r times that day's night consumption estimates midday demand without PV. The gap to the actual midday withdrawal is the power covered by the panels, averaged over the summer as Δ. A linear regression fitted on labeled customers converts Δ into kWp. This estimate is then combined, through a geometric mean, with the labeled customers' median capacity (169.5 kWp) to limit large errors on unusual plants.

2.6 Bonus task – extensions

We attempted the optional task with the same model as Task 2: after the commissioning date we look for a *second* downward step, at least 60 days after the first and 60 days before the end of the data. Before searching, the Task 1 detector checks whether PV was already present before the first step, so that an extension is not mistaken for the original installation. We output at most the three clearest steps over the whole test set. With a single labelled extension in the development data, this part is not validated on real cases (see *Ideas that did not work*).

3 Results

Each customer is predicted by a pipeline whose thresholds and coefficients are refitted on the other 39 (leave-one-customer-out). The **Final pipeline** PV detection operates on the last 2 years, while date and kWp on all 4 available years (2022–2025). The final robustness check works on 5 rounds of 2-year windows obtained from the whole 4 year time-lapse.

	F1	D	P	Score
Baseline (all PV, mid-series date, median kWp)	0.710	0.045	0.415	54.58
Previous: detection on the whole series	0.905	0.665	0.452	78.49
Final pipeline	0.952	0.632	0.429	**81.82**
Robustness check (final pipeline), 5 ∗ 2-year windows	0.909	0.608	0.439	77.93 ± 2.42

true \ pred.	PV	no
PV	20	2
no PV	0	18

Table 1: Left: leave-one-customer-out on the 40 labelled customers, with the official score F1 × (70 + 15D + 15P). D and P are the mean date and capacity scores over correctly detected PV customers: 1 for an exact answer, falling linearly to 0 at 30 days or 50% error. Right: confusion matrix of the final version, full series.

What is behind the 81.8:

- **Detection** finds 20 of 22 PV systems, with no false alarm among the 18 customers without PV. Moving detection to the last two years changed exactly two customers (one plant found, one false alarm removed); the other 38 predictions are identical. D and P drop slightly because the newly found plant gets a very wrong date and size. The 2 missed plants are tiny compared with the customer's load (night load 1.6 to 5.4 times the plant's kWp) and look like non-PV customers on every feature we tried. Fitting everything on all 40 customers gives 82.00, almost the same as leave-one-out: the models do not overfit.
- **Dates** are the strong part: median error 1 day, 13 of 20 within a week. 5 are more than 30 days off.
- **Capacity** is the weak part: median error 22%, 9 of 20 within 20%, but 8 above 50%. The median alone already gives $P = 0.415$.

4 Challenges

4.1 Problems we hit and how we solved them

- **Dirty data.** Some rows had included irrelevant customer ids that we were required to remove. We kept only `POD` ids, set the rare negative kWh to 0 and left missing values as NaN to not confuse them with potential PV.
- **A 7.9 GB file.** Our first sort in DuckDB ran out of memory and left an empty file. We capped memory at 4 GB, wrote to a temporary file renamed only on success, and sorted per customer later.
- **Midday saturation.** For the median PV customer, about half of the summer midday intervals are at zero withdrawal: the meter cannot go below zero, so large plants look smaller than they are.
- **Will the threshold hold on new customers?** We ran detection on the 1147 unlabelled customers (only as a check, never for training): 34% are flagged as PV. An unsupervised cut (Otsu) on the same feature lands at 1.08

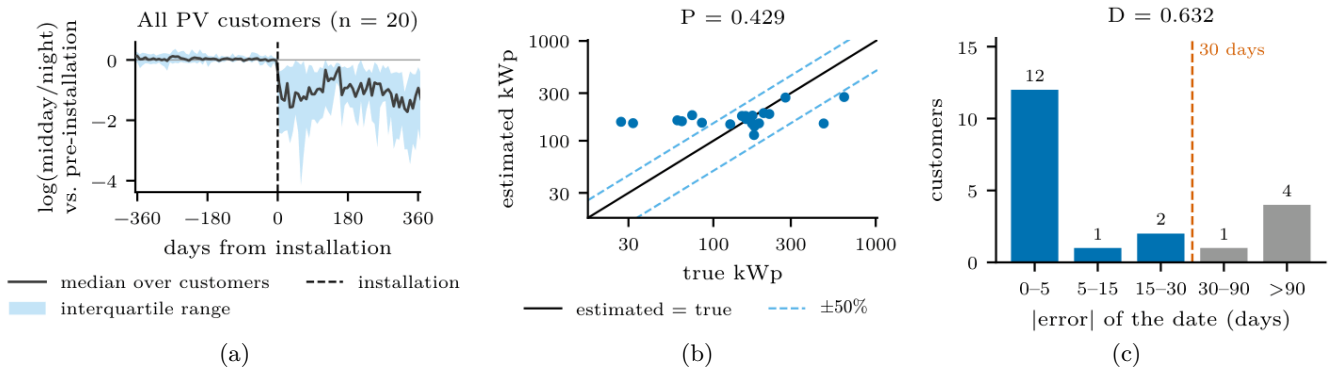


Figure 2: **(a)** Midday-to-night ratio of the 20 PV customers with at least six months of data before and after installation, aligned on the installation date (day 0) and scaled by each customer's own pre-installation level (hence the narrow band before day 0). Line: weekly median; band: middle half of customers. After installation the ratio drops sharply and stays low; how far depends on the system size compared with the customer's consumption. **(b)** Estimated vs. true capacity for the 20 correctly detected PV customers (leave-one-out); dashed lines: ±50%, beyond which the capacity score is 0. Estimates cluster around 150–200 kWp: the model captures the typical size better than individual differences. **(c)** Absolute date error for the same 20 customers; errors above 30 days score 0. 15 of 20 are within 30 days, 12 within 5 days.

against our 0.85: our threshold sits on the tail of the no-PV group, not in a gap between groups, so we estimate about 7% false alarms among no-PV customers. Raising it would lose one real PV system in our data, so we kept it and report the risk.

- **Malformed rows in the hidden test file.** One bad timestamp used to crash the whole script. Now the reader skips bad rows, keeps every customer id, and if everything fails it still writes the files.

4.2 Ideas that did not work

Every idea faced the same leave-one-out test, with the bar to beat written down beforehand (F1 on 2-year windows, scores on full series).

- **Detection.** Average summer dip: F1 0.894. Decision trees and a random forest: at best a tie with our rule (0.952). Logistic regression on four features: 78.70 against 78.49 for the rule, but it wins in only 48.5% of bootstrap resamples, i.e. noise. Calibrating the thresholds on full 4-year series: 70.14, because summers before and after the installation get mixed. Six new features in 22 variants (e.g. year-over-year by month, Sunday-only dip spread) never rose above noise level; the Sunday version recovered the same two customers as our final change, an independent sign that this gain is not luck.
- **Pseudo-labels.** We labelled the unlabelled customers where three models (rule, logistic, tree) agreed and retrained: 11 variants, not a single prediction changed. The extra labels made the model more confident without moving its boundary.
- **Stricter second pass.** On unlabelled customers `rec_drop` fires 39 times, and its mirrored version (which can only fire by chance) 40 times: on new customers it is mostly noise. Every stricter variant we tried lost the only recent plant it recovers (installed 75 days before the end of the data). We kept it: we expect it to add about as many hits as false alarms on the test.
- **Date.** A generic change-point detector (`ruptures`): $D = 0.523$. A step that also changes the seasonal shape: 0.570. Our constant step: 0.597. A two-step version reached 0.628, but the whole gain came from one customer, so we did not adopt it.
- **Capacity.** Only our geometric mean (0.437) beat simply predicting the median ($P = 0.415$). Ridge regression on five features gave 0.378, night consumption 0.286, and the peak drop 0.212, since it measures the household load rather than the plant.
- **Extensions.** Comparing months with ones a year earlier found the date of the only labeled extension within 6 days, but its signal was inside the noise of customers without extensions: a sunnier month moves the signal as much as doubling the plant. Our step method reaches F1 0.29–0.50 on synthetic extensions and 0 on the real one.

Honest caveat. Thresholds and coefficients are refitted in leave-one-out, but we chose the features and methods while looking at all 40 customers, so 81.8 is probably slightly optimistic: one customer is worth about 2 points.

5 AI agents and LLM usage

Several of us used Claude Code (Anthropic) as a coding assistant. It helped write and review code and tests, wrote some validation scripts, ran experiments we asked for, and searched for edge cases on synthetic data. Ideas and decisions were discussed by the team; the acceptance rules were set by us before running. A team rule limited what the agent could see: per-customer aggregates, metrics and predictions compared with labels, never the raw consumption series. Every significant agent task and every discarded idea is logged in `docs/LOG.md`. No LLM is part of `inference.py`. This report was written by us.